

# **SIMATS ENGINEERING**

## **ASSESSMENT TOOL 1**

**COURSE CODE: CSA51 COURSE NAME: Cryptography and Network Security**

### **COURSE OUTCOME COVERED**

**CO4:** *Implement best security practices for IP, email, and web service using PGP, S/MIME for enhancing email Security.CO (BL3, BL6)*

*Assessment Tool Weightage: 40%*

**QUESTIONS: 5 TOTAL MARKS: 25**

### **Annexure A**

#### ***Coding Challenge***

#### ***Code of Conduct:***

*I Anto princyA(192411071) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***

**Annexure A**  
***Coding Challenge***

1. Write a program to simulate the IPSec Authentication Header (AH) process that generates and verifies a message authentication code (MAC) for IP packet integrity.

```
import hmac
```

```
import hashlib
```

```
# Function to generate MAC
```

```
def generate_mac(packet, key):
```

```
    return hmac.new(key.encode(), packet.encode(), hashlib.sha256).hexdigest()
```

```
# Function to verify MAC
```

```
def verify_mac(packet, key, mac):
```

```
    expected_mac = generate_mac(packet, key)
```

```
    return hmac.compare_digest(expected_mac, mac)
```

```
# Example usage
```

```
if __name__ == "__main__":
```

```
    # Simulated IP packet (header + payload)
```

```
    ip_packet = "SRC=192.168.1.1;DST=192.168.1.2;DATA=Hello IPSec"
```

```
    secret_key = "shared_secret"
```

```
# Sender generates MAC
```

```
mac = generate_mac(ip_packet, secret_key)
```

```
print("Generated MAC:", mac)
```

```
# Receiver verifies MAC
```

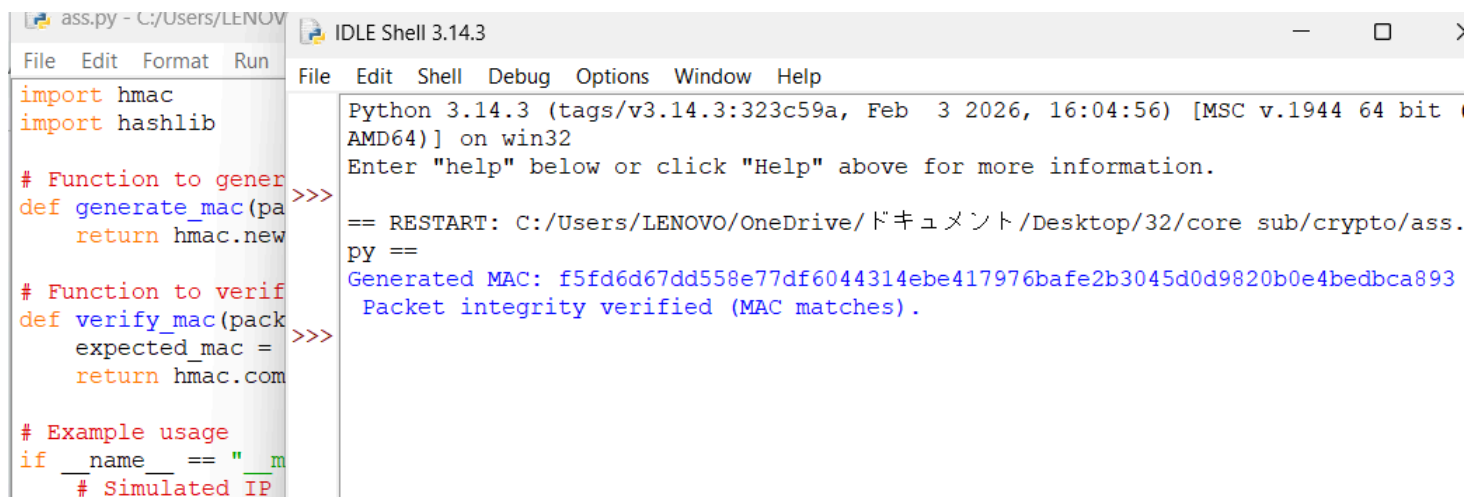
```
if verify_mac(ip_packet, secret_key, mac):
```

```
    print(" Packet integrity verified (MAC matches).")
```

```
else:
```

```
    print("Packet integrity failed (MAC mismatch).")
```

O/P:



The image shows a screenshot of a Python script named 'ass.py' and its execution output in the IDLE Shell 3.14.3. The script defines two functions: 'generate\_mac' and 'verify\_mac'. The 'generate\_mac' function takes an IP packet and a secret key as input and returns a new MAC. The 'verify\_mac' function takes an IP packet, a secret key, and an expected MAC as input and returns a comparison result. The script also includes an example usage section with a simulated IP packet and a secret key. The output of the script shows the generated MAC and a confirmation that the packet integrity is verified.

```
ass.py - C:/Users/LENOV
File Edit Format Run
import hmac
import hashlib

# Function to gener
def generate_mac(pa
    return hmac.new

# Function to verif
def verify_mac(pack
    expected_mac =
    return hmac.com

# Example usage
if __name__ == "__m
    # Simulated IP
```

```
IDLE Shell 3.14.3
File Edit Shell Debug Options Window Help
Python 3.14.3 (tags/v3.14.3:323c59a, Feb  3 2026, 16:04:56) [MSC v.1944 64 bit
AMD64] on win32
Enter "help" below or click "Help" above for more information.

>>> == RESTART: C:/Users/LENOVO/OneDrive/ドキュメント/Desktop/32/core sub/crypto/ass.
py ==
Generated MAC: f5fd6d67dd558e77df6044314ebe417976bafe2b3045d0d9820b0e4bedbca893
Packet integrity verified (MAC matches).
```

2. Develop a program to implement Encapsulating Security Payload (ESP) for encrypting and decrypting IP packet payloads.

```
payload = input("Enter payload: ")
```

```
key = int(input("Enter key: "))
```

```
encrypted = ""
```

```
for ch in payload:
```

```
    encrypted += chr(ord(ch) + key)
```

```
print("Encrypted payload:", encrypted)
```

```
decrypted = ""
```

```
for ch in encrypted:
```

```
    decrypted += chr(ord(ch) - key)
```

```
print("Decrypted payload:", decrypted)
```

O/P:

A screenshot of a Python IDE window. The title bar shows 'File Edit Shell Debug Options Window Help'. The main text area displays the following text: 'Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64]', 'Enter "help" below or click "Help" above for more information.', and a separator line '===== RESTART: C:/Users/DeLL/OneDrive/Documents/coding.py ====='. Below the separator, the program's execution is shown: '>>>' followed by 'Enter payload: hello', 'Enter key: 3', 'Encrypted payload: khood', and 'Decrypted payload: hello'. The prompt '>>>' appears again at the end.

```
File Edit Shell Debug Options Window Help
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64]
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/DeLL/OneDrive/Documents/coding.py =====
Enter payload: hello
Enter key: 3
Encrypted payload: khood
Decrypted payload: hello
>>>
```

3. Design a program to manage Security Associations (SA) in IPSec, including creation, lookup, and deletion operations.

```
class SecurityAssociation:
```

```
    def __init__(self, spi, key, algo):
```

```
        self.spi = spi        # Security Parameter Index
```

```
        self.key = key        # Secret key
```

```
        self.algo = algo      # Algorithm (e.g., AES, HMAC)
```

```
class SA_Manager:
```

```
    def __init__(self):
```

```
        self.sa_table = {}    # Dictionary to store SAs
```

```
    def create_sa(self, spi, key, algo):
```

```
        if spi in self.sa_table:
```

```
            print("SA with SPI already exists.")
```

```
        else:
```

```
            self.sa_table[spi] = SecurityAssociation(spi, key, algo)
```

```
print(f"SA created: SPI={spi}, Algo={algo}")
```

```
def lookup_sa(self, spi):
```

```
    sa = self.sa_table.get(spi)
```

```
    if sa:
```

```
        print(f"Found SA: SPI={sa.spi}, Algo={sa.algo}, Key={sa.key}")
```

```
    else:
```

```
        print("SA not found.")
```

```
def delete_sa(self, spi):
```

```
    if spi in self.sa_table:
```

```
        del self.sa_table[spi]
```

```
        print(f"SA with SPI={spi} deleted.")
```

```
    else:
```

```
        print("SA not found.")
```

```
# Example usage
```

```
if __name__ == "__main__":
```

```
    manager = SA_Manager()
```

```
    # Create SAs
```

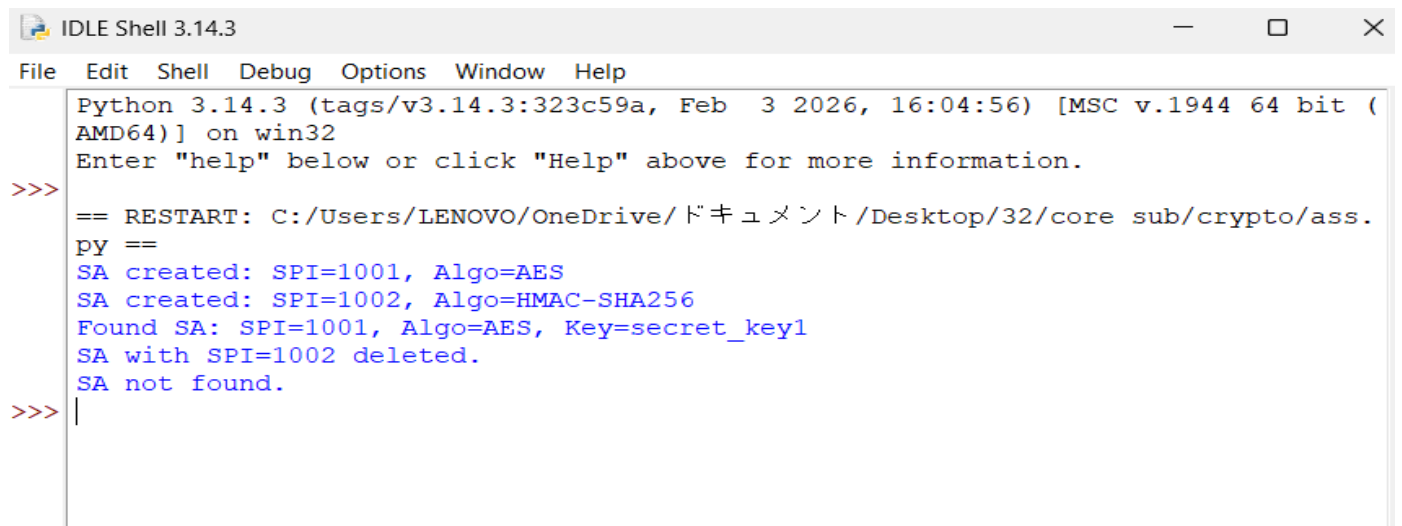
```
    manager.create_sa(1001, "secret_key1", "AES")
```

```
    manager.create_sa(1002, "secret_key2", "HMAC-SHA256")
```

```
    # Lookup SA
```

```
    manager.lookup_sa(1001)
```

O/P:



```
IDLE Shell 3.14.3
File Edit Shell Debug Options Window Help
Python 3.14.3 (tags/v3.14.3:3223c59a, Feb 3 2026, 16:04:56) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> == RESTART: C:/Users/LENOVO/OneDrive/ドキュメント/Desktop/32/core sub/crypto/ass.py ==
SA created: SPI=1001, Algo=AES
SA created: SPI=1002, Algo=HMAC-SHA256
Found SA: SPI=1001, Algo=AES, Key=secret_key1
SA with SPI=1002 deleted.
SA not found.
>>> |
```

4. Implement a simplified Pretty Good Privacy (PGP) system for secure email communication using public key encryption and digital signatures.

Code:

```
message = input("Enter the message: ")
```

```
key = int(input("Enter the key: "))
```

```
encrypted = ""
```

```
for ch in message:
```

```
    encrypted += chr(ord(ch) + key)
```

```
print("Encrypted message:", encrypted)
```

```
decrypted = ""
```

```
for ch in encrypted:
```

```
    decrypted += chr(ord(ch) - key)
```

```
print("Decrypted message:", decrypted)
```

```
IDLE Shell 3.14.2
File Edit Shell Debug Options Window Help
Python 3.14.2 (tags/v3.14.2:df79316, Dec 5 2025, 17:18:21) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/DeLL/OneDrive/Documents/coding.py =====
Enter the message: hello
Enter the key: 3
Encrypted message: kloor
Decrypted message: hello
>>>
```

5. Create a program to classify emails as spam or legitimate using email header analysis and content-based filtering techniques.

### Code:

```
import re
```

```
class EmailFilter:
```

```
    def __init__(self):
```

```
# Simple spam indicators

self.spam_headers = ["X-Spam-Flag", "Received-SPF: fail"]

self.spam_keywords = ["win money", "free offer", "click here", "lottery", "urgent"]

def classify(self, headers, body):

    # Header analysis

    for h in self.spam_headers:

        if h.lower() in headers.lower():

            return "SPAM (header rule)"

    # Content-based filtering

    body_lower = body.lower()

    for kw in self.spam_keywords:

        if re.search(kw, body_lower):

            return "SPAM (content rule)"

    return "LEGITIMATE"
```

# Example usage

```
if __name__ == "__main__":
```

```
    headers = "From: unknown@spam.com\nX-Spam-Flag: YES"
```

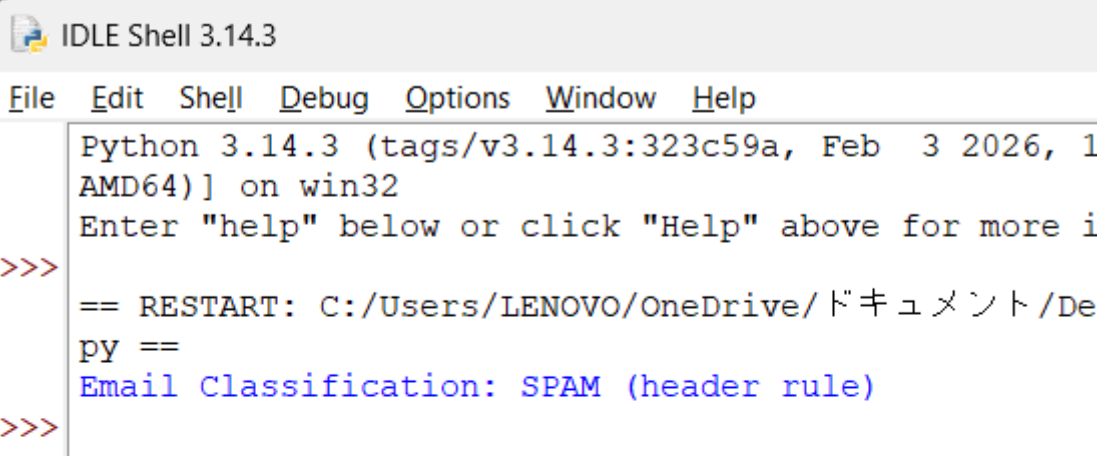
```
    body = "Congratulations! You win money, click here to claim."
```

```
    filter = EmailFilter()
```

```
    result = filter.classify(headers, body)
```

```
    print("Email Classification:", result)
```

**O/P:**



```
IDLE Shell 3.14.3
File Edit Shell Debug Options Window Help
Python 3.14.3 (tags/v3.14.3:323c59a, Feb  3 2026, 1
AMD64)] on win32
Enter "help" below or click "Help" above for more i
>>>
== RESTART: C:/Users/LENOVO/OneDrive/ドキュメント/De
py ==
Email Classification: SPAM (header rule)
>>>
```

**RUBRICS FOR CALCULATION:**

| <b>Q.<br/>No</b>             | <b>Understandi<br/>ng of<br/>Concepts (25)</b> | <b>Experime<br/>nta l<br/>Design<br/>(30)</b> | <b>Use of<br/>Tools (20)</b> | <b>Analysis<br/>of<br/>Results<br/>(15)</b> | <b>Docume<br/>nta tion<br/>(10)</b> | <b>Total<br/>Score<br/>(out of<br/>100)</b> |
|------------------------------|------------------------------------------------|-----------------------------------------------|------------------------------|---------------------------------------------|-------------------------------------|---------------------------------------------|
| <b>1</b>                     | / 4                                            | / 4                                           | / 4                          | / 4                                         | / 4                                 |                                             |
| <b>2</b>                     | / 4                                            | / 4                                           | / 4                          | / 4                                         | / 4                                 |                                             |
| <b>3</b>                     | / 4                                            | / 4                                           | / 4                          | / 4                                         | / 4                                 |                                             |
| <b>4</b>                     | / 4                                            | / 4                                           | / 4                          | / 4                                         | / 4                                 |                                             |
| <b>5</b>                     | / 4                                            | / 4                                           | / 4                          | / 4                                         | / 4                                 |                                             |
| <b>OVERALL RUBRICS VALUE</b> |                                                |                                               |                              |                                             |                                     |                                             |
|                              | / 4                                            | / 4                                           | / 4                          | / 4                                         | / 4                                 |                                             |
|                              | <b>/ 25</b>                                    | <b>/ 30</b>                                   | <b>/ 20</b>                  | <b>/ 15</b>                                 | <b>/ 10</b>                         | <b>/ 100</b>                                |